



## Today's agenda

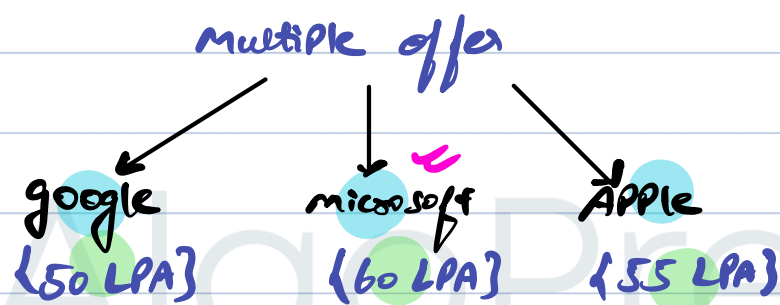
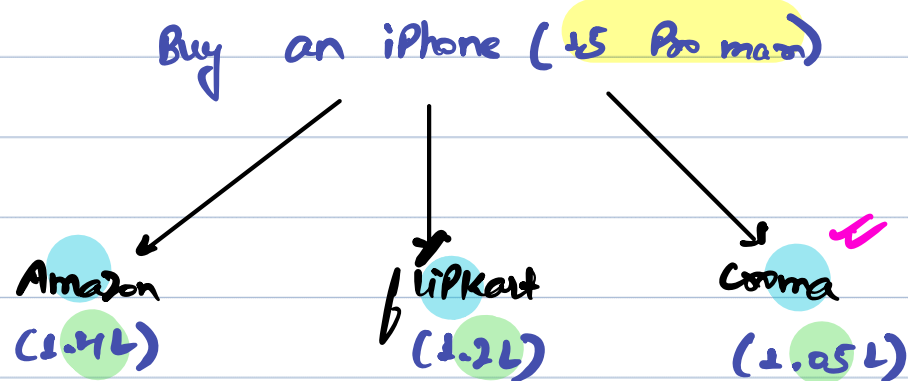
- ↳ Greedy Intro
- ↳ fractional Knapsack
- ↳ Greedy Properties
- ↳ Activity Selection
- ↳ Job Scheduling



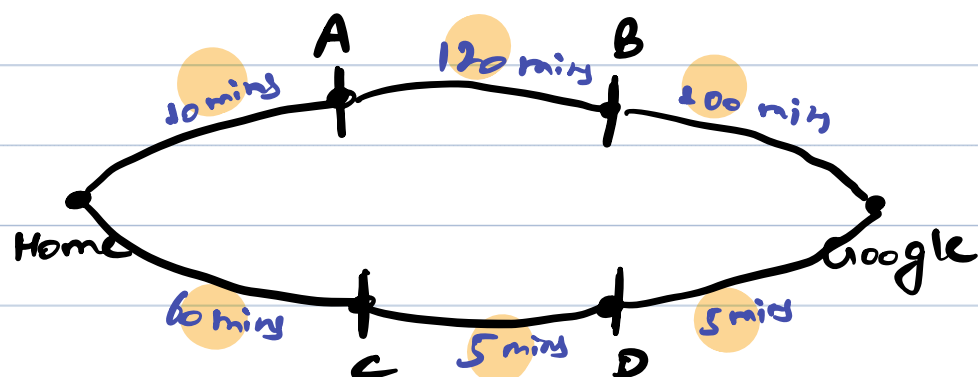
# AlgoPrep



## Greedy



→ greedy is an approach to solve problems by making locally optimal choice at each step.





## Q) Fractional Knapsack

↳ you can consume  $K$  Kg of food, you can eat any integral amount of item. Max Protein you can get?

| vegetables        | Eating Complete item<br>Protein gained<br>/kg | Protein /kg | eat on the basis of max Protein |
|-------------------|-----------------------------------------------|-------------|---------------------------------|
| Tomato → 0 20 Kg  | 200P → 10P * 20 = 200P                        |             | Onion → 50 Kg                   |
| Apples → 1 15 Kg  | 180P → 12P * 15 = 180P                        |             | ↓<br>250P                       |
| Onion → 2 50 Kg   | 250P → 5P                                     |             | ↳ Tomato → 20kg                 |
| Chicken → 3 10 Kg | 150P → 15P * 10 = 150P                        |             | ↓<br>200P                       |
| Potato → 4 25 Kg  | 200P → 8P * 8 = 64P                           |             | ↓<br>450P                       |
| Mango → 5 12 Kg   | 132P → 11P * 12 = 132P                        |             |                                 |
| Seafood → 6 5 Kg  | 100P → 20P * 5 = 100P                         |             |                                 |
|                   | 826P                                          |             |                                 |

(1.1 + 0.1)

$$\frac{\text{this.P} * 0.1}{0.1} - 0.1 * \text{this.w}$$

~~this.w \* 0.1~~

$$\frac{\text{this.P}}{\text{this.w}} - \frac{0.1}{0.1}$$

$$\text{return } (\text{this.P} * 0.1) - (0.1 * \text{this.w})$$

Class Pair implements Comparable {

int w;

int P;

double PPU;

int compareTo (Pair o) {

return this.PPU - o.PPU;

}



## Wgued code

```
int fractionalKnapsack (int wt[N], int profit[N], int K) {  
    Pair[] items = new Pair[N];  
    for (int i=0; i<N; i++) {  
        items[i] = new Pair (wt[i], profit[i],  $\frac{\text{profit}[i]}{\text{wt}[i]}$ );  
    }
```

Arrays.Sort (items);

double ans=0;  
for (int i=N-1; i>=0; i--) {

if (items[i].w <= K) {  
 ans = ans + items[i].p;  
 K = K - items[i].w;

}

else {

ans = ans + (K \* items[i].p/w);  
break;

}

}

return ans;

}

T.C:  $O(N \log N)$   
S.C:  $O(N)$



Arrays.Sort (items);

Wt(i): 20 15 50 10 25 12 5

Profit(i): 200 180 250 150 200 132 100

double ans=0;

for (int i = N-1; i >= 0; i--) {

if (items[i].w <= K) {

ans = ans + items[i].P;

K = K - items[i].w;

}

else {

ans = ans + (K \* items[i].PPV);

break;

}

return ans;

items[i]:  $\begin{pmatrix} 20 \\ 200 \\ 10 \end{pmatrix}$   $\begin{pmatrix} 15 \\ 180 \\ 12 \end{pmatrix}$   $\begin{pmatrix} 50 \\ 250 \\ 5 \end{pmatrix}$   $\begin{pmatrix} 10 \\ 150 \\ 15 \end{pmatrix}$   $\begin{pmatrix} 25 \\ 200 \\ 8 \end{pmatrix}$   $\begin{pmatrix} 12 \\ 132 \\ 11 \end{pmatrix}$   $\begin{pmatrix} 5 \\ 100 \\ 20 \end{pmatrix}$

$\begin{pmatrix} 50 \\ 250 \\ 5 \end{pmatrix}$   $\begin{pmatrix} 25 \\ 200 \\ 8 \end{pmatrix}$   $\begin{pmatrix} 30 \\ 200 \\ 10 \end{pmatrix}$   $\begin{pmatrix} 12 \\ 132 \\ 11 \end{pmatrix}$   $\begin{pmatrix} 15 \\ 180 \\ 12 \end{pmatrix}$   $\begin{pmatrix} 10 \\ 150 \\ 15 \end{pmatrix}$   $\begin{pmatrix} 5 \\ 100 \\ 20 \end{pmatrix}$

K=20

ans = 0 + 100 + 150 + 180 + 132 + 200 + 64

K = 20 - 5 = 15 - 10 = 5 - 15 = 40 - 12 = 28 - 20

= 8 - 8 = 0



AlgoPrep



\* Greedy Properties

Binary Search  
DP

↳ ① for min/max related Problems.

↳ ② based on what Parameter we want to apply greedy.

↳ for every Parameter prove it logically or discard with one counter example.



AlgoPrep

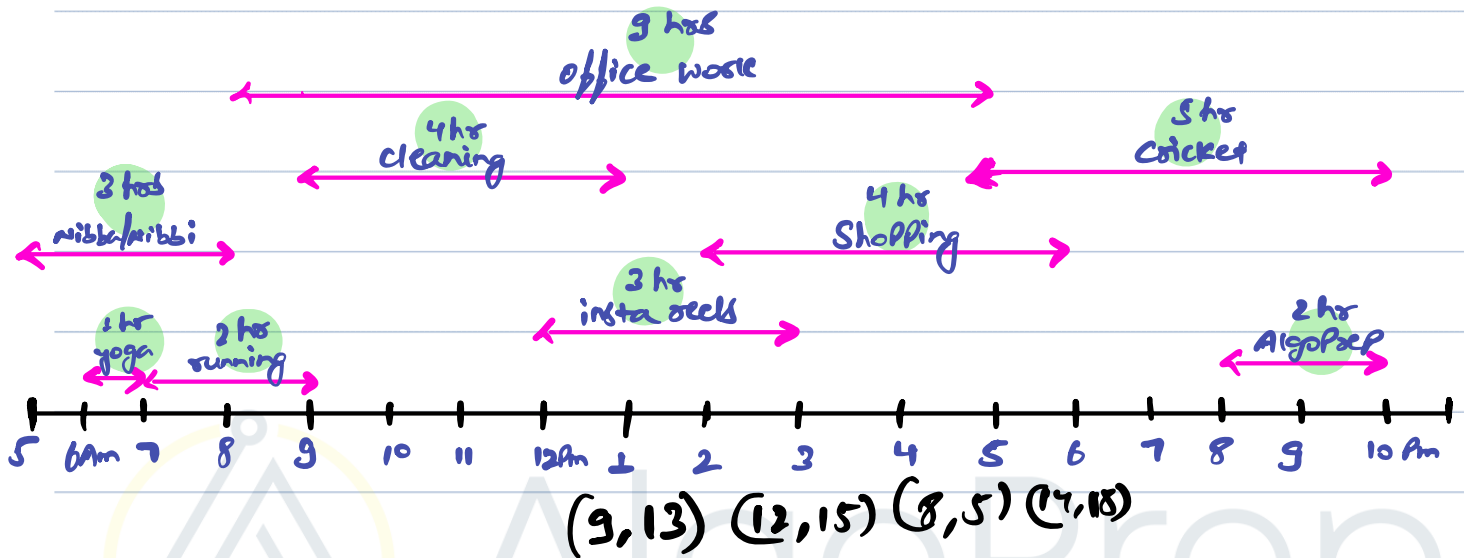


## Activity Selection

↳ max count of task you can do.

① Start a task, we need to complete

② At any Point of time, Single task.



① min duration task first X X

↳ yoga, running, AlgoraeP, insta reels  
↳ 4 tasks

② based on starting time X X

③ ending early (on the basis of end time)

↳ yoga, running, cleaning, Shopping, AlgoraeP  
↳ 5 tasks

## Correctness:

A: 8 AM  
8 AM  
n tasks

B: 6 AM  
6 AM  
n tasks



```
class Pair {  
    int s;  
    int e;  
}
```

//Sudo code

```
int ActivitySelection (int start[N], int end[N]) {
```

```
    Pair [ ] arr = new int [N];
```

```
    for (int i=0; i<N; i++) {  
        arr[i] = new Pair (start[i], end[i]);  
    }
```

```
    Arrays.sort (arr);
```

sort on the basis of endtime.

T.C:  $O(N \log N)$

S.C:  $O(N)$

//Pick all the non-overlapping task.

```
    int ans = 1;
```

```
    int currenttime = arr[0].e;
```

```
    for (int i=1; i<N; i++) {  
        if (arr[i].s >= currenttime) {  
            ans++;  
            currenttime = arr[i].e;  
        }  
    }
```

```
    return ans;
```

```
}
```



11 Pick all the non-overlapping tasks.

0 1 2 3 4 5 6  
(6,7) (5,8) (7,9) (9,13) (12,15) (8,17) (14,18)

int ans = 1;

int currenttime = arr[0][1];

```
for (int i = 1; i < N; i++) {  
    if (arr[i][0] >= currenttime) {  
        ans++;  
        currenttime = arr[i][1];  
    }  
}
```

ans = 1 2 3 4

currenttime = 18 18 18

return ans;

Break till 9:50 PM



## Job Scheduling

↳ Given  $N$  tasks to complete.

→ Deadline assigned for each task, day on or before we can do task.

→ Payment assigned to each task.

→ On any given day we can perform only 1 task and each task take 1 day to finish.

→ find max payment we can get.

Ex1:

| Job | deadline | Payment |
|-----|----------|---------|
| a   | 3        | 100 ✓   |
| b   | 1        | 19      |
| c   | 2        | 27 ✓    |
| d   | 1        | 25 ✓    |
| e   | 3        | 20      |

Parameter  
↓  
Payment  
 $O(N^2)$

d c a  
1 2 3

(1)  $100 + 27 + 25 = 152$

Ex2:

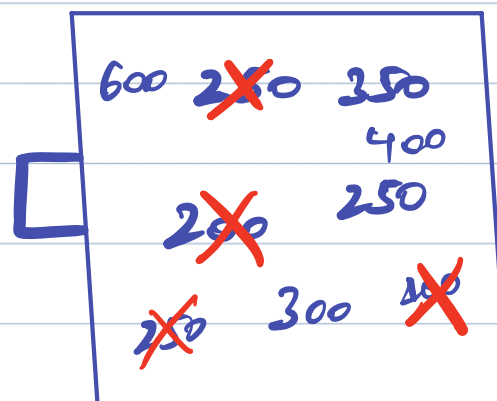
| Job | deadline | Payment |
|-----|----------|---------|
| a   | 3        | 5       |
| b   | 1        | 1       |
| c   | 3        | 6       |
| d   | 2        | 3       |
| e   | 3        | 9       |

a c e

→ Sort on the basis of deadline



|         | 0   | 1   | 2   | 3   | 4   | 5   | 6   | 7   | 8   | 9   | 10  |
|---------|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| dead:   | 1   | 1   | 1   | 2   | 2   | 2   | 4   | 4   | 5   | 5   | 5   |
| Profit: | 250 | 200 | 350 | 200 | 150 | 250 | 300 | 250 | 100 | 600 | 400 |



min heap



AlgoPrep



// Pseudo code

```
class Pair {  
    int d;  
    int p;  
}
```

```
int jobScheduling (int deadline[], int payment[])
```

```
Pair[] arr = new Pair[N];
```

```
for (int i=0; i<N; i++) {  
    arr[i] = new Pair(deadline[i], payment[i]);  
}
```

```
Arrays.sort(arr);
```

T.C:  $O(N \log N)$

S.C:  $O(N)$

```
PriorityQueue<Integer> pq = ...
```

```
for (int i=0; i<N; i++) {  
    if (arr[i].d > pq.size()) {  
        pq.add(arr[i].p);  
    }
```

```
    else {
```

```
        if (arr[i].p > pq.peek()) {  
            pq.remove();  
            pq.add(arr[i].p);  
        }
```

```
    }
```

```
}
```

// return sum of all elements of heap

```
}
```